Taller de Desarrollo de Aplicaciones · IUA

Dietética Los Mirasoles

Informe de entrega final · ciclo 2026

Kevin Ojea y Juan Garcia

Comisión A · Ing. Gustavo Marcelo Macarone

3 de noviembre de 2026

https://github.com/kojea415-bot/TDA-ECommerce-LosMirasoles

EquipoOjea_Informe.pdf · EquipoOjea_Codigo.txt

No hay número de equipo.

1. Identificación

El proyecto es la tienda web Dietética Los Mirasoles, un comercio de San Justo. El equipo está formado por Kevin Ojea y Juan Garcia. La materia es Taller de Desarrollo de Aplicaciones, comisión A, a cargo del Ing. Gustavo Marcelo Macarone.

2. Problema y alcance

El local vende en el mostrador y resuelve los pedidos a distancia por WhatsApp. No hay un catálogo público, el stock y los precios se controlan a mano, y un cliente con una restricción alimentaria no puede filtrar la oferta sin consultar. El sistema ofrece una tienda web para armar el pedido y un panel para que los dueños actualicen el catálogo y sigan los pedidos. El cobro es simulado. El comprobante sale por correo real.

Usuarios de esta entrega: visitante, que recorre el catálogo sin cuenta; cliente registrado, que confirma la compra; administrador, que gestiona productos y pedidos y no usa la tienda.

Incluye esta entrega, completo salvo las salvedades de la sección 6:

- Catálogo público por categorías planas, ficha, etiquetas, filtros combinables y búsqueda por nombre.
- Carrito en memoria, con cantidades, tope de stock y subtotal.
- Registro, ingreso y cierre de sesión. La compra exige sesión.
- Checkout con retiro o envío, costo por código postal, cobro simulado y confirmación con número de pedido.
- Comprobante por SMTP. Si el correo falla, el pedido queda y `mail_ok` queda en falso.
- Panel de productos y de pedidos, con permisos distintos de los del cliente.

El catálogo cargado tiene 32 productos.

3. Requerimientos

Cada requisito queda identificado en la tabla, con su prioridad y el estado de esta entrega. La columna Historia vincula los que corresponden a una historia de prioridad alta. «Completo» significa que el comportamiento de la sección 6 está en la aplicación. «Parcial» no debe leerse como un hecho cerrado.

ID	Función	Prioridad	Estado	Historia
RF-01	Catálogo público por categoría, sin cuenta	Alta	Parcial: hay categorías, no subcategorías	HU-01
RF-02	Ficha con nombre, descripción, imagen, precio, unidad, aptitud, nutrición y disponibilidad	Alta	Completo	HU-02
RF-03	Etiquetas sin TACC, vegano y sin lactosa	Alta	Completo	HU-03
RF-04	Filtros de categoría y aptitud, combinables, sin recargar	Alta	Completo	HU-03
RF-05	Búsqueda por nombre, sin distinguir mayúsculas ni tildes, ordenada por relevancia	Media	Parcial: filtra el nombre ya cargado; no ordena por relevancia	—
RF-06	Sin stock visible y no agregable	Alta	Completo	HU-02
RF-08	Alta, cantidad, quita y subtotal del carrito	Alta	Completo	HU-04
RF-11	Revalidar stock al confirmar	Alta	Completo en la base. La prueba automática simula el 409	HU-06
RF-12	Retiro o envío	Alta	Completo	HU-07
RF-13	Validar el código postal contra las zonas	Alta	Parcial: una zona conocida muestra su costo; un código desconocido no rechaza el envío y el costo queda en 0	HU-07

ID	Función	Prioridad	Estado	Historia
RF-14	Mostrar el costo antes de confirmar. Retiro en cero	Alta	Completo	HU-07
RF-17	Nombre, teléfono, correo y domicilio si hay envío	Alta	Completo	HU-08
RF-18	Cobro simulado, sin datos de tarjeta	Alta	Completo como resultado aprobado, rechazado o pendiente. No hay selector de medio de pago	HU-08
RF-19	Aprobado prepara y descuenta; rechazado no graba; pendiente espera y no descuenta	Alta	Completo	HU-08
RF-20	Confirmación con número, ítems, total y modalidad	Alta	Completo. Recargar la pantalla no conserva el número	HU-08
RF-21	Comprobante SMTP. Si falla, el pedido queda	Alta	Completo	HU-13
RF-22	Alta con nombre, correo y contraseña. El correo repetido se rechaza	Alta	Completo	HU-05
RF-23	Ingreso y cierre de sesión	Alta	Completo	HU-05
RF-28	Panel solo para el administrador	Alta	Completo	HU-09
RF-29	Alta, edición, despublicar y eliminar productos	Alta	Completo	HU-10
RF-31	Precio y stock desde la fila	Alta	Completo	HU-11
RF-33	Listado de pedidos con filtros y búsqueda	Alta	Completo	HU-12
RF-34	Cambio de estado con fecha	Alta	Completo	HU-12

Criterio observable de las historias construidas, en una línea: el visitante ve el catálogo publicado; sin stock no se agrega; los filtros se combinan en la página; el carrito respeta el stock y el subtotal; si el producto ya está en el carrito, el catálogo y la ficha usan el mismo − / cantidad / +; sin sesión no aparece «Ir a pagar» y «Seguir comprando» lleva al ingreso; el correo repetido no crea otra cuenta; al confirmar, si falta stock no hay pedido; el código 1754 muestra envío $ 2.500 y el retiro muestra $ 0; aprobado deja el pedido en preparación y vacía el carrito; rechazado no graba y no explica la causa; pendiente graba y no descuenta; el comprobante llega o `mail_ok` queda en falso; el cliente que abre el panel ve el aviso; el administrador entra a Pedidos y no recorre la tienda.

Requisitos no funcionales

ID	Cómo se verificó
RNF-01	El circuito de compra se revisó a 360 px de ancho, sin desplazamiento horizontal. Cypress corre en 360×740. Las figuras de este informe muestran la misma tienda en una pantalla de escritorio de 1440×900.
RNF-02	No verificado. No se midió el catálogo sobre una red 4G.
RNF-03	El alta de producto es un solo formulario y el precio y el stock se editan en la fila. No se cronometró el uso por una persona ajena al equipo.
RNF-04	La contraseña la guarda Supabase Auth con hash. El panel y el stock se controlan con políticas de fila y con la función de confirmación, no solo ocultando botones. No se auditó el algoritmo de hash por fuera de Auth.
RNF-05	El checkout no tiene campo de tarjeta. El test de pago aprobado revisa que el cuerpo enviado no lleve precio ni dato de tarjeta. Verificado en Cypress el 29/10/2026.
RNF-06	No verificado. No hay consentimiento ni baja de cuenta según la Ley 25.326.
RNF-07	No verificado. No hay medición de disponibilidad.

ID	Cómo se verificó
RNF-08	No verificado en Chrome, Firefox, Safari y Edge. La corrida manual y Cypress usaron Chromium.
RNF-09	No verificado. No hay prueba de carga.
RNF-10	La función bloquea los productos y rechaza la segunda venta si no alcanza el stock. No se ejecutaron dos confirmaciones simultáneas en esta entrega.
RNF-11	No verificado. No se probó una restauración.
RNF-12	El código está separado por pantallas, contexto y acceso a datos, y el repositorio tiene README de instalación. Ver sección 6.
RNF-13	`pedido_cambios` guarda el estado y la fecha. No guarda el usuario ni el precio o el stock anterior.
RNF-14	No verificado contra una pauta de accesibilidad. Hay etiquetas en botones e inputs.
RNF-15	La interfaz y los precios visibles están en español y en pesos, con el formato de `es-AR` (por ejemplo $ 1.200 y la fecha 29/10/26). El selector nativo de fecha del panel sigue el idioma del navegador: en la captura se ve `mm/dd/aaaa`.

4. Análisis

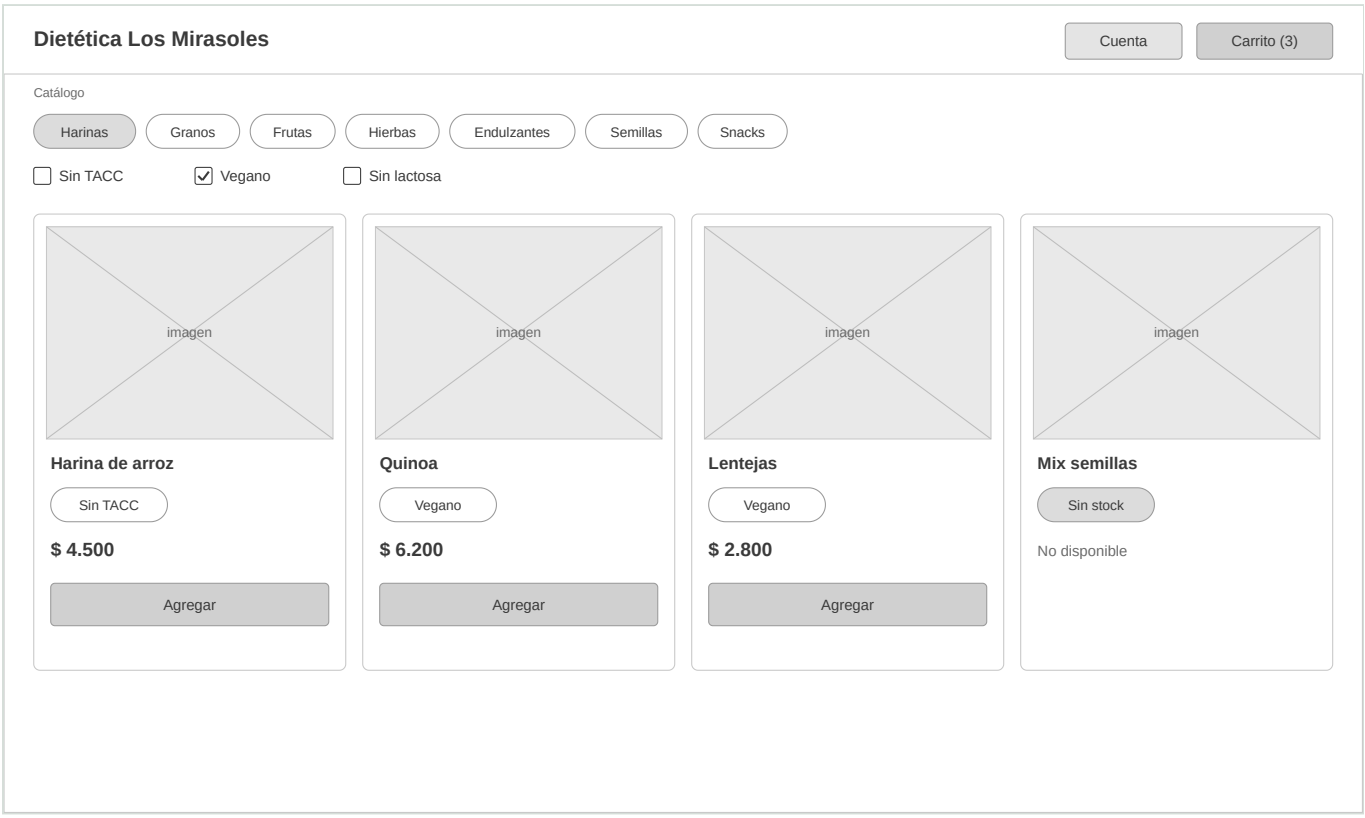
El backlog de esta entrega es el corte de prioridad alta. El tablero de Jira no reemplaza esta tabla: quien lea el informe no necesita abrirlo. El estado es el del 29/10/2026.

Historia	Tareas concretas	Estado	Responsable
HU-01 Catálogo	Listado público, categorías, enlace a la ficha	Hecho, sin subcategorías	Juan Garcia y Kevin Ojea
HU-02 Ficha y sin stock	Ficha y bloqueo de agregado	Hecho	Juan Garcia y Kevin Ojea
HU-03 Filtros	Categoría y aptitud combinables	Hecho	Juan Garcia y Kevin Ojea
HU-04 Carrito	Agregar, cantidad, quitar, vaciar, subtotal	Hecho	Juan Garcia y Kevin Ojea
HU-05 Cuenta	Alta, ingreso, salida, correo repetido	Hecho	Juan Garcia y Kevin Ojea
HU-06 Stock al confirmar	Relectura, rechazo y descuento en la transacción	Hecho	Kevin Ojea
HU-07 Entrega	Retiro, envío, costo de zona	Hecho, con la salvedad del código desconocido	Kevin Ojea
HU-08 Checkout	Datos, simulación, confirmación	Hecho	Kevin Ojea
HU-13 Correo	SMTP y `mail_ok`	Hecho	Kevin Ojea
HU-09 a HU-12 Panel	Acceso, alta, precio, stock, estados	Hecho	Kevin Ojea
Pruebas de cliente	Cypress de catálogo, carrito, ingreso y checkout	Hecho, 11 pruebas	Juan Garcia y Kevin Ojea

Reglas que condicionan el comportamiento: el precio del pedido se copia al confirmar; el cliente no envía el precio; sin sesión no se confirma; el rechazo no crea fila; el pendiente no descuenta stock; el stock se descuenta al entrar en preparación, listo para retiro, despachado o entregado, y vuelve si el pedido sale de esos estados; «listo para retiro» solo aplica a un retiro y «despachado» solo a un envío; el rol no se cambia desde la cuenta del cliente; un código postal fuera de la tabla no bloquea el envío.

Riesgos tratados en el corte: el catálogo puede publicarse por tandas (hay 32 productos); el cobro no pide tarjeta; si Gmail falla el pedido queda; dos confirmaciones del último unidad se resuelven con un bloqueo en la base, aunque ese caso no se ejecutó; el costo de una zona conocida se muestra antes de pagar.

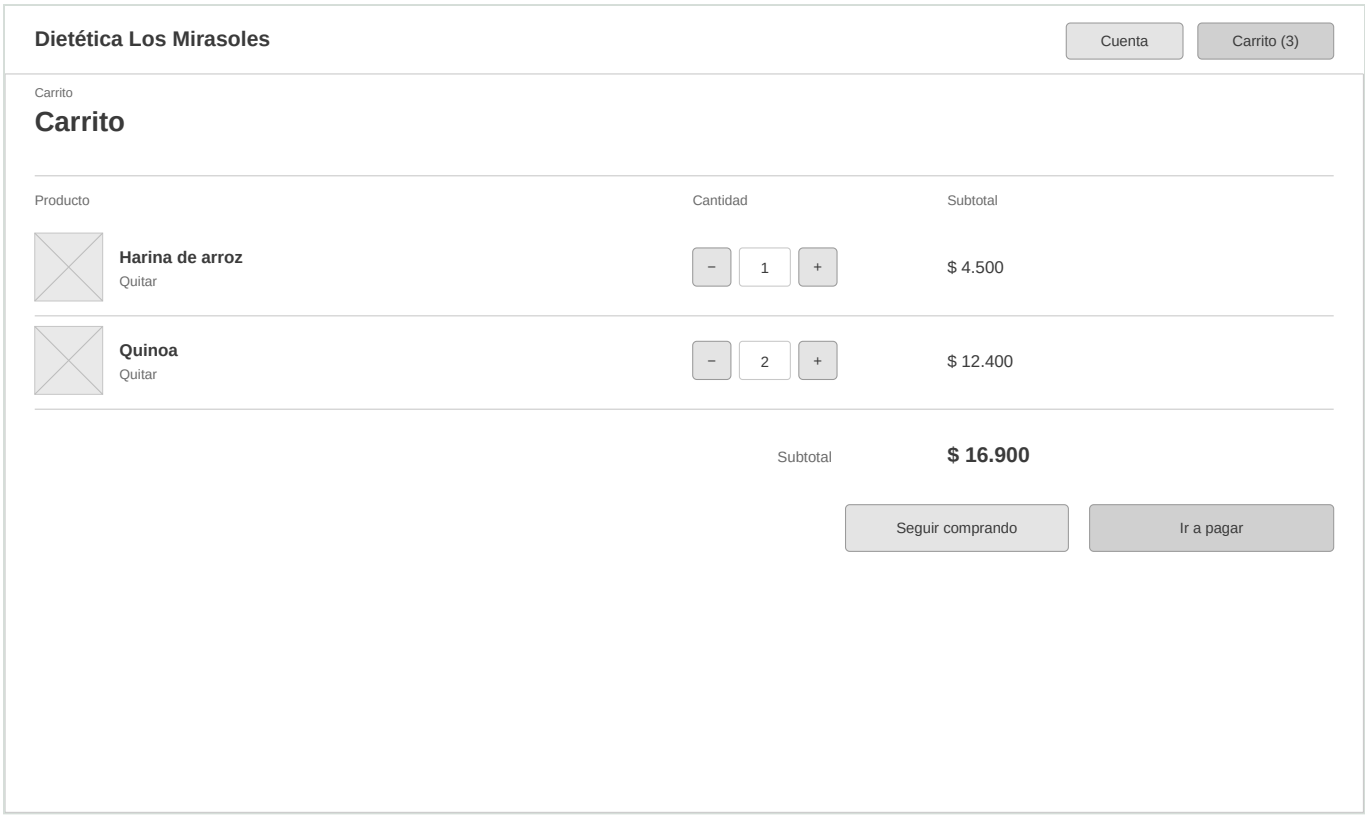
Bocetos de baja fidelidad de la fase de análisis:



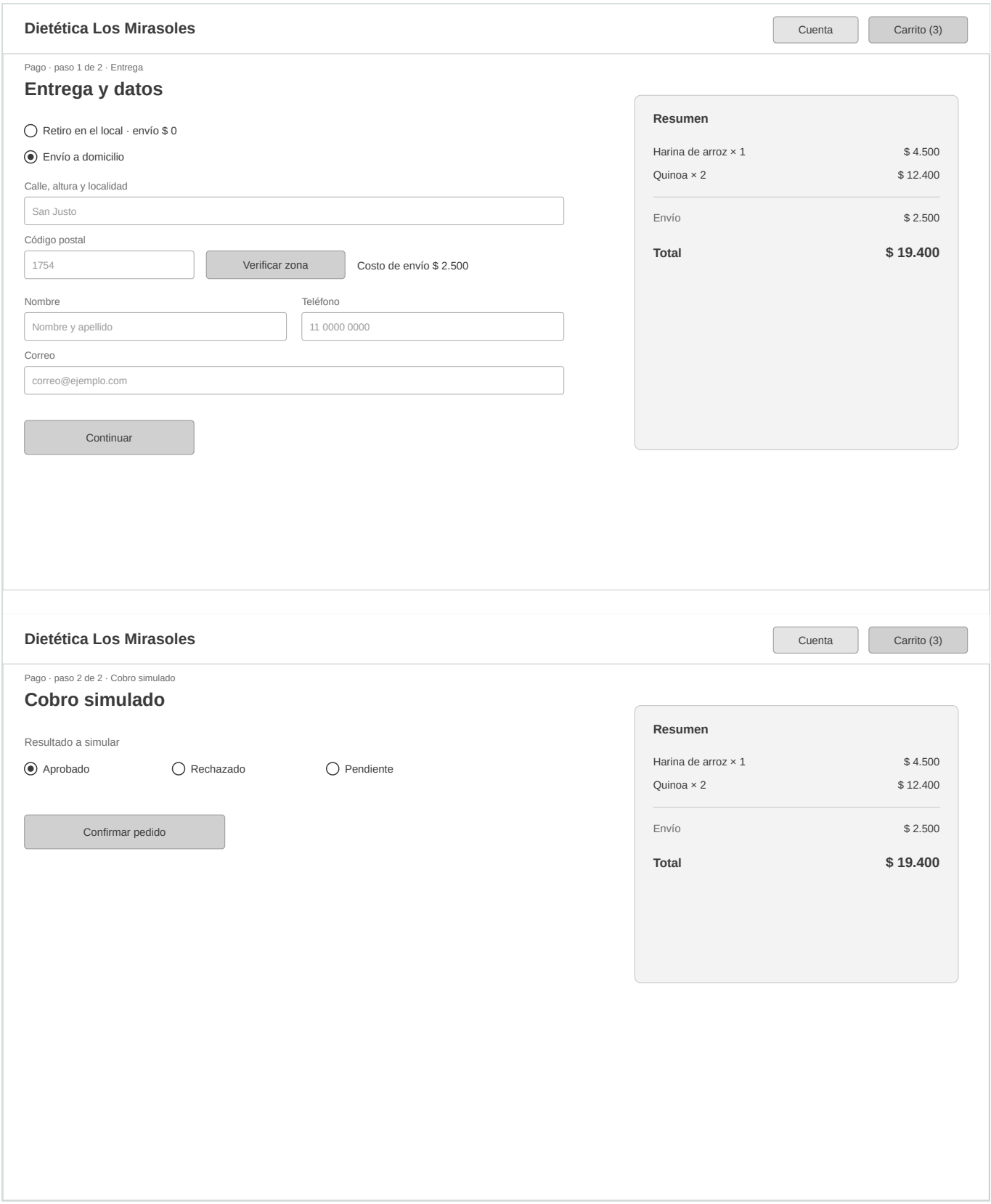
Boceto 1. Catálogo, con categorías, aptitud y el botón Agregar.



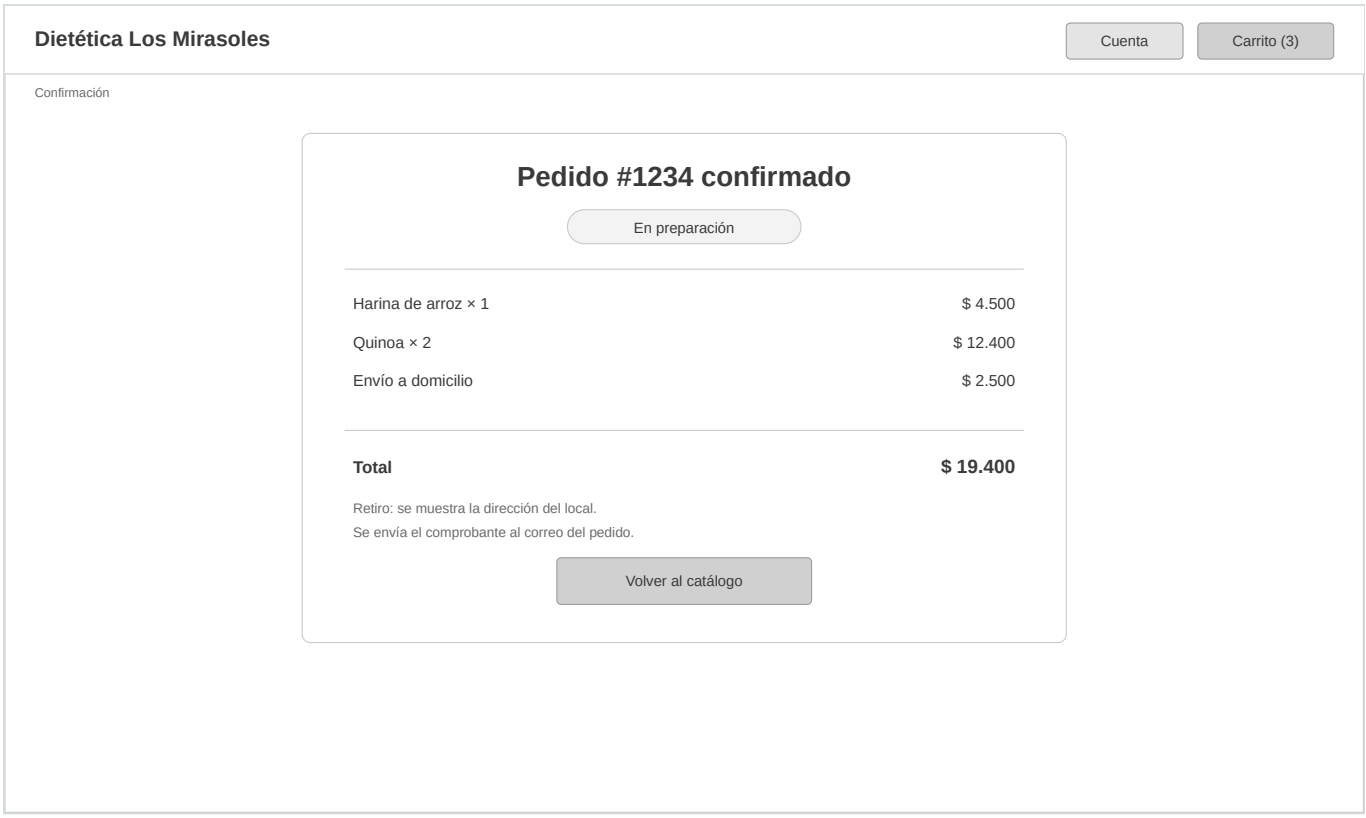
Boceto 2. Ficha de producto.



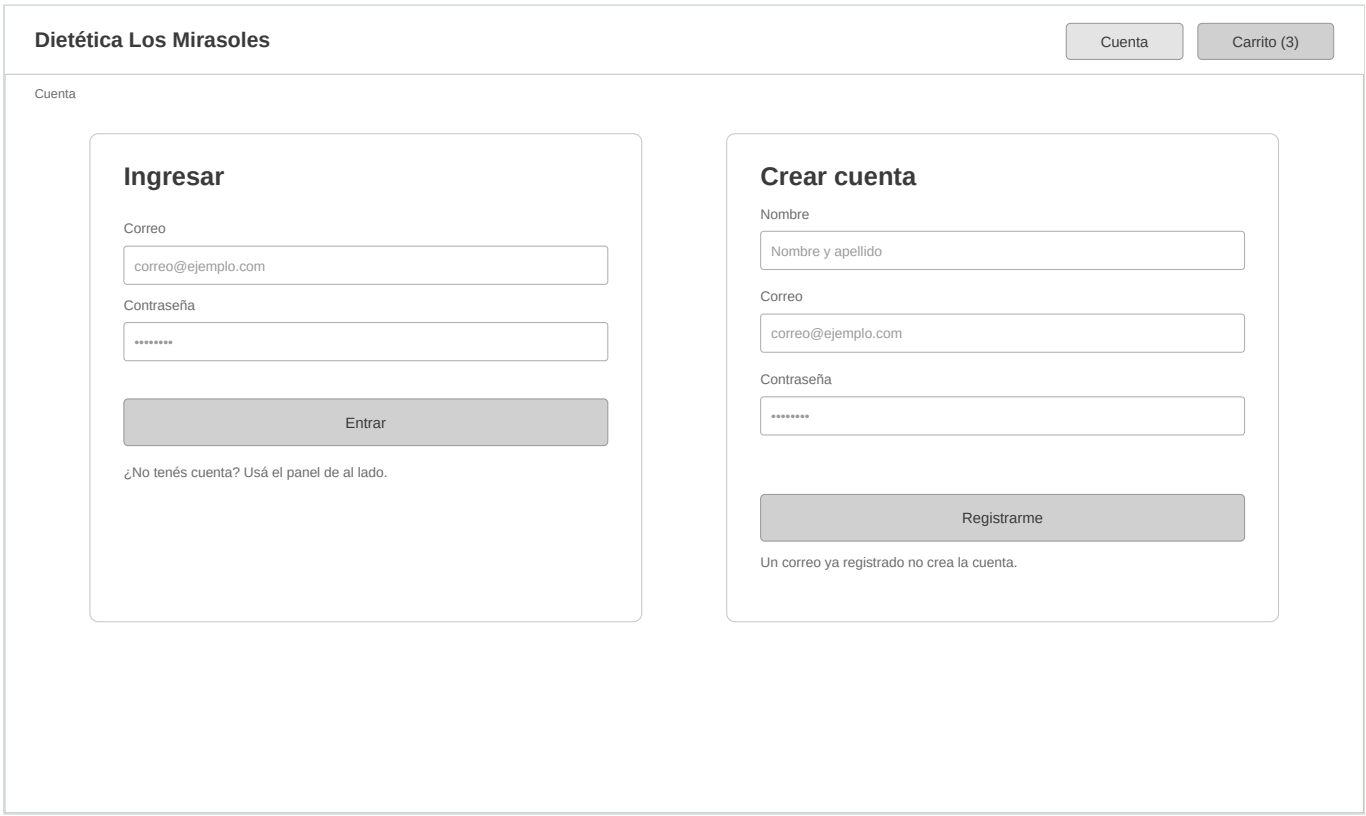
Boceto 3. Carrito.



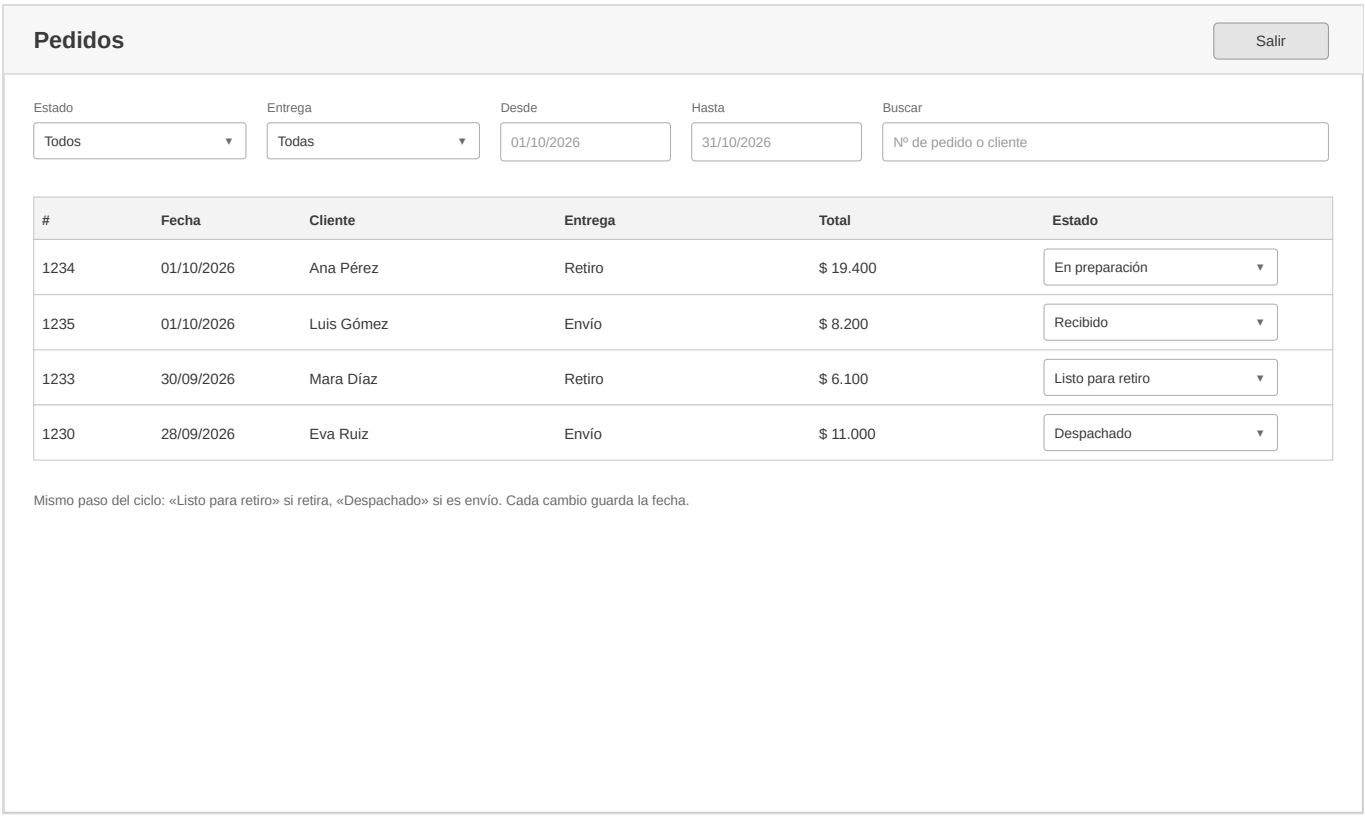
Boceto 4. Checkout, en dos pasos.



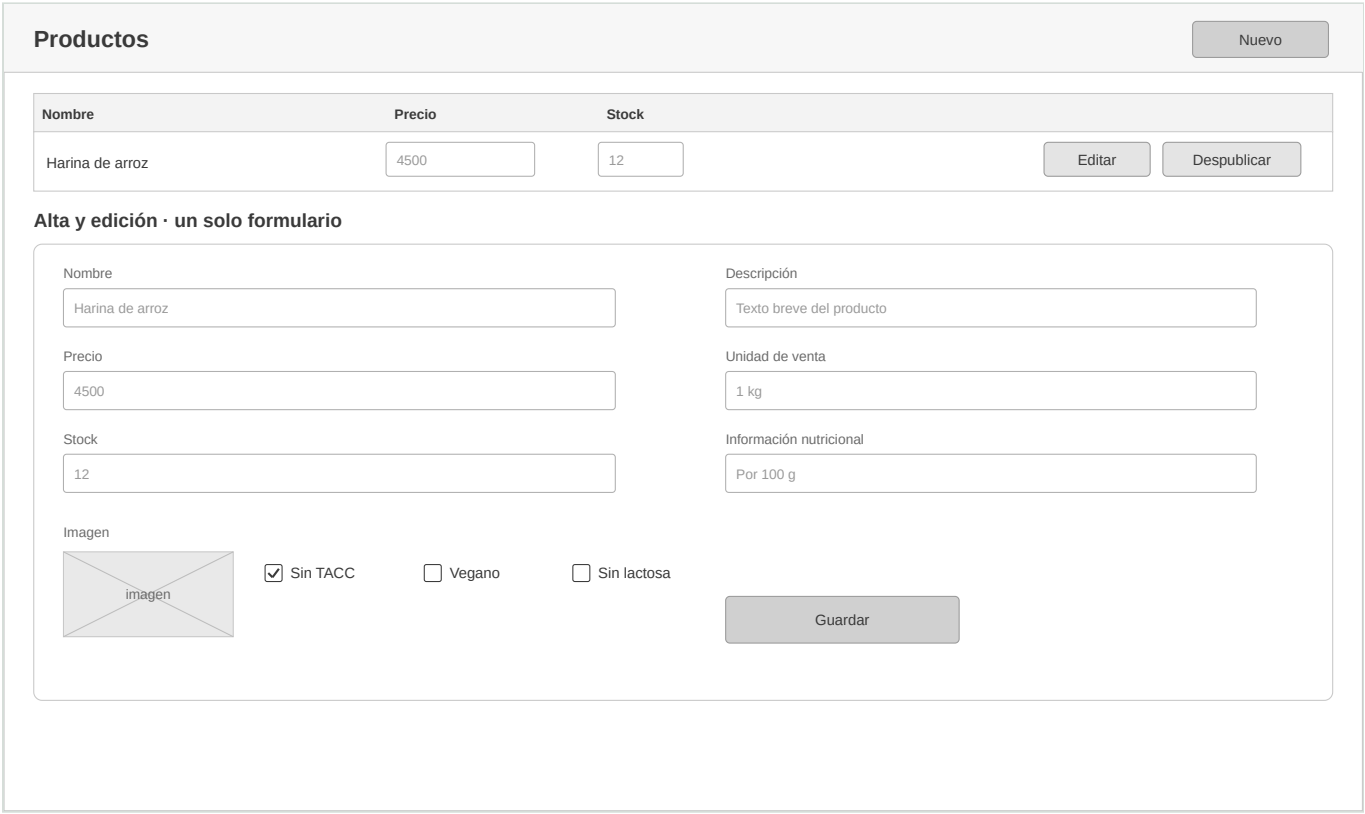
Boceto 5. Confirmación.



Boceto 6. Ingreso y registro.



Boceto 7. Pedidos del administrador.



Boceto 8. Productos del administrador.

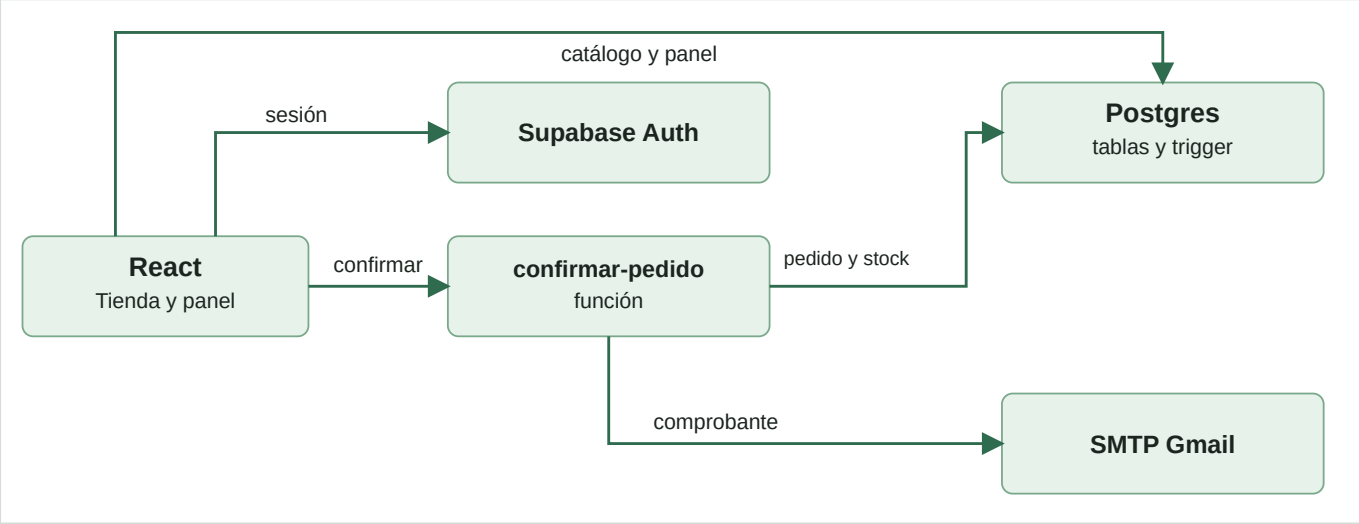
Diferencias entre estos bocetos y la tienda:

- El catálogo tiene búsqueda por nombre. El boceto 1 no la muestra.
- Si el producto ya está en el carrito, el catálogo y la ficha muestran − / cantidad / +. Los bocetos 1 y 2 muestran Agregar.

- Sin sesión, el carrito no muestra «Ir a pagar». El boceto 3 lo muestra junto a «Seguir comprando». En la tienda, «Seguir comprando» lleva al ingreso y después al catálogo.
- Un cobro rechazado conserva el carrito, no crea pedido y muestra un texto fijo, sin el motivo. El boceto 5 vuelve al paso 2 y muestra ese motivo.
- La tienda es verde y blanca. Los bocetos son grises.

5. Diseño

La tienda es una sola aplicación React. El navegador lee el catálogo en Postgres a través de Supabase. La sesión la resuelve Supabase Auth. Confirmar un pedido no escribe las tablas desde el navegador: llama a la función `confirmar-pedido`, que valida al usuario y ejecuta `confirmar_pedido` dentro de una transacción. Después de grabar, la misma función manda el comprobante por SMTP. Si el correo falla, la transacción no se deshace. Así el cliente no puede fijar un precio ni un stock. El panel actualiza productos y el estado del pedido en Postgres; el trigger de esa base mueve el stock.



Esquema de arquitectura. El navegador confirma por la función. Esa función graba el pedido y manda el comprobante. El panel escribe en Postgres, y el trigger mueve el stock.

Módulos y archivos:

Módulo	Responsabilidad	Archivos
Tienda	Decide si la ruta es de cliente o de administrador	`src/App.jsx`
Catálogo y ficha	Listado, filtros, búsqueda y detalle	`src/pages/Catalogo.jsx`, `src/pages/Ficha.jsx`
Carrito	Estado en memoria	`src/context/CartContext.jsx`, `src/pages/Carrito.jsx`, `src/components/ControlCantidad.jsx`
Cuenta	Alta, ingreso y rol	`src/pages/Ingreso.jsx`, `src/context/AuthContext.jsx`
Checkout	Entrega, simulación y confirmación	`src/pages/Pago.jsx`, `src/pages/Confirmacion.jsx`
Panel	Productos, pedidos y puerta de acceso	`src/pages/admin/`, `src/components/PuertaAdmin.jsx`
Datos y correo	Tablas, transacción y SMTP	`supabase/schema.sql`, `supabase/functions/confirmar-pedido/index.ts`

El precio y el costo de envío se copian en el pedido. Las políticas de acceso están en `supabase/schema.sql`. Las imágenes están en el depósito `productos` de Supabase, no en el repositorio.

Tabla	Clave	Relación
`categorias`	`id`	
`productos`	`id`	`categoria_id` hacia `categorias`
`zonas_cobertura`	`codigo_postal`	
`perfiles`	`id`	`id` hacia `auth.users`
`pedidos`	`id`	`perfil_id` hacia `perfiles`
`pedido_items`	`id`	`pedido_id` hacia `pedidos`; `producto_id` hacia `productos`
`pedido_cambios`	`id`	`pedido_id` hacia `pedidos`

Versiones declaradas en `package.json` y usadas para desarrollar y probar: Node 24.13.1, React 19.2.8, react-router-dom 7.18.4, Vite 8.3.0, `@supabase/supabase-js` 2.118.0, Cypress 16.2.0.

Se eligió Supabase porque el entregable necesita Postgres, cuentas y archivos sin operar un servidor propio, y una función para que el precio y el stock no los decida el navegador. React alcanza para las pantallas de un equipo de dos personas. El SMTP de Gmail es la única integración externa pedida: el comprobante es real y el cobro sigue simulado. No se integró una pasarela porque el alcance prohíbe guardar datos de tarjeta.

6. Implementación

Árbol de lo que se entrega como código, sin dependencias ni imágenes:

```
index.html
package.json
.env.example
vite.config.js
cypress.config.js
src/                  pantallas, contexto y estilos
cypress/e2e/          pruebas del cliente
supabase/schema.sql
supabase/functions/confirmar-pedido/index.ts
README.md
```

Función	Estado	Qué falta, si es parcial
Catálogo, ficha, filtros, carrito, cuenta, checkout, correo, panel	Completa, con las salvedades ya dichas	RF-01 sin subcategorías. RF-05 sin orden por relevancia. RF-13 no rechaza un código desconocido

Para ejecutarla hace falta Node 24 y npm. En la carpeta del proyecto:

1. Copiar `.env.example` a `.env` y completar `VITE_SUPABASE_URL` y `VITE_SUPABASE_ANON_KEY` con los valores del proyecto de Supabase. Esos valores no están en este informe ni en el archivo de código.
2. `npm install`
3. `npm run dev`. El inicio correcto es la dirección local que imprime Vite, en este equipo `http://127.0.0.1:5173`, y el título de la tienda en el encabezado.
4. Las pruebas de cliente, con la tienda ya levantada: `npm run cy:run`.

Sin el proyecto de Supabase la tienda no lee productos ni inicia sesión. El esquema de referencia está en `supabase/schema.sql`, pero no alcanza para clonar los 32 productos ni sus imágenes: eso está en el proyecto alojado. Sin `GMAIL_USER` y `GMAIL_APP_PASSWORD` en los secretos de la función, el pedido se graba y el comprobante no sale. Esas dos variables no se versionan.

Cuentas de demostración, las dos en `/ingreso`. Son cuentas ficticias del trabajo práctico.

Cuenta	Correo	Contraseña
Cliente	`cliente.prueba@mail.com`	`Mirasoles.prueba`
Administrador	`administrador_losmirasoles@mail.com`	`Mirasoles.admin`

El cliente ve la tienda. El administrador entra a Pedidos. Productos está en la misma barra.

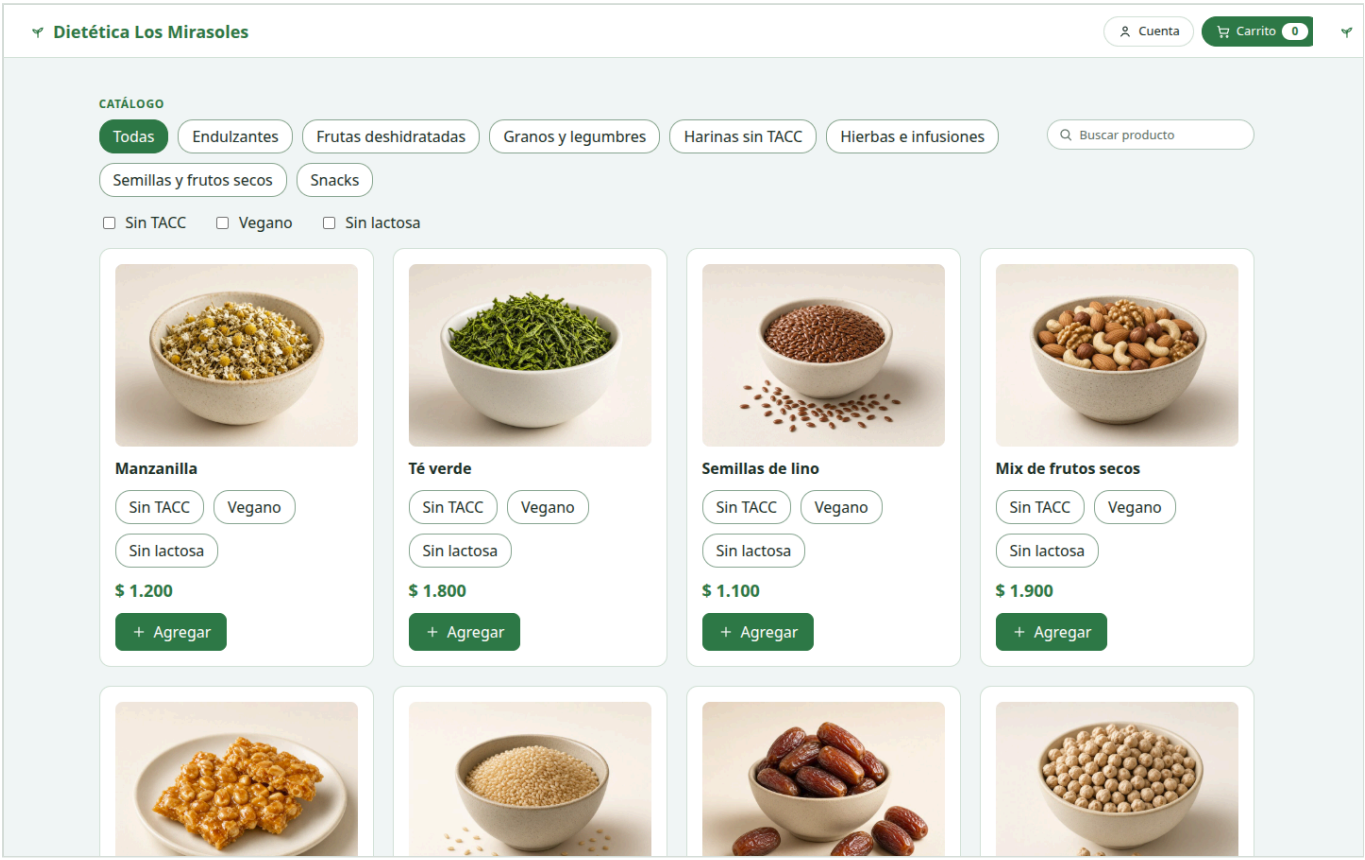


Figura 1. Sin sesión se abrió el catálogo. Se ven las categorías, la búsqueda y la tarjeta de Manzanilla con precio y «Agregar». RF-01, RF-03 y RF-04. La búsqueda de RF-05 está en la misma pantalla.

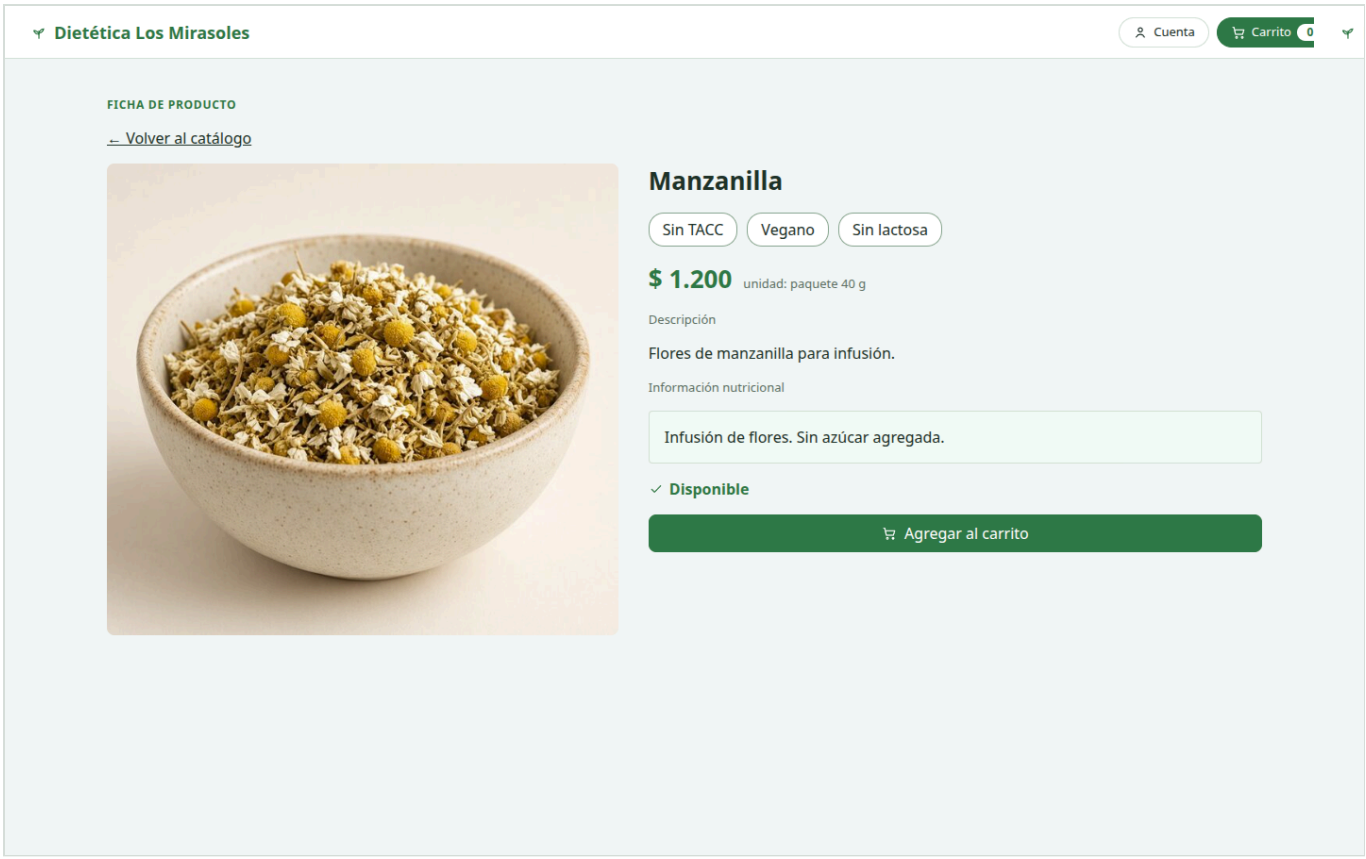


Figura 2. Se abrió la ficha de Manzanilla. Muestra imagen, etiquetas, precio, unidad, descripción, nutrición y el botón para agregar. RF-02.

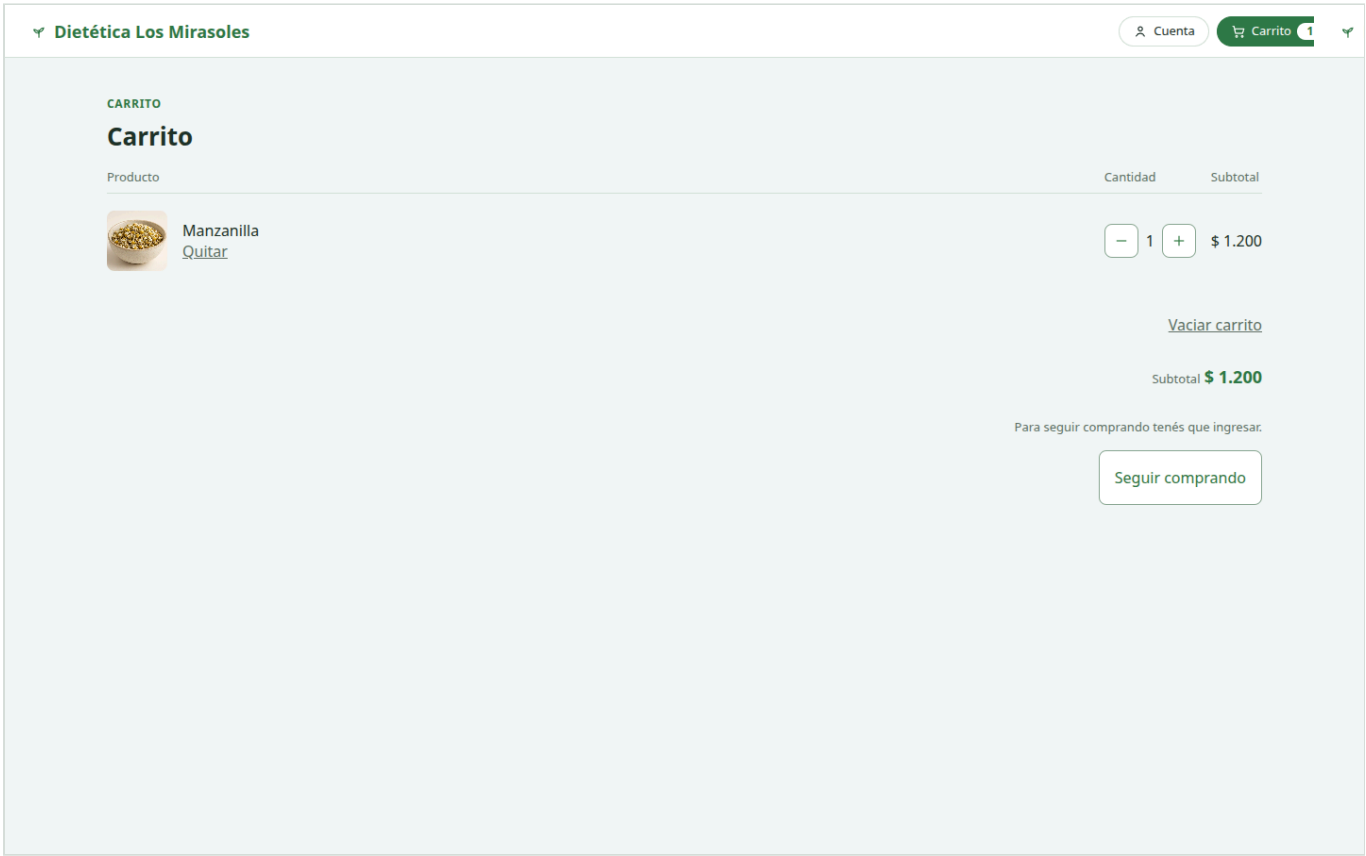


Figura 3. Se agregó una unidad. El carrito muestra cantidad, subtotal $ 1.200 y el aviso de que para seguir comprando hay que ingresar. No aparece «Ir a pagar». RF-08 y RF-23.

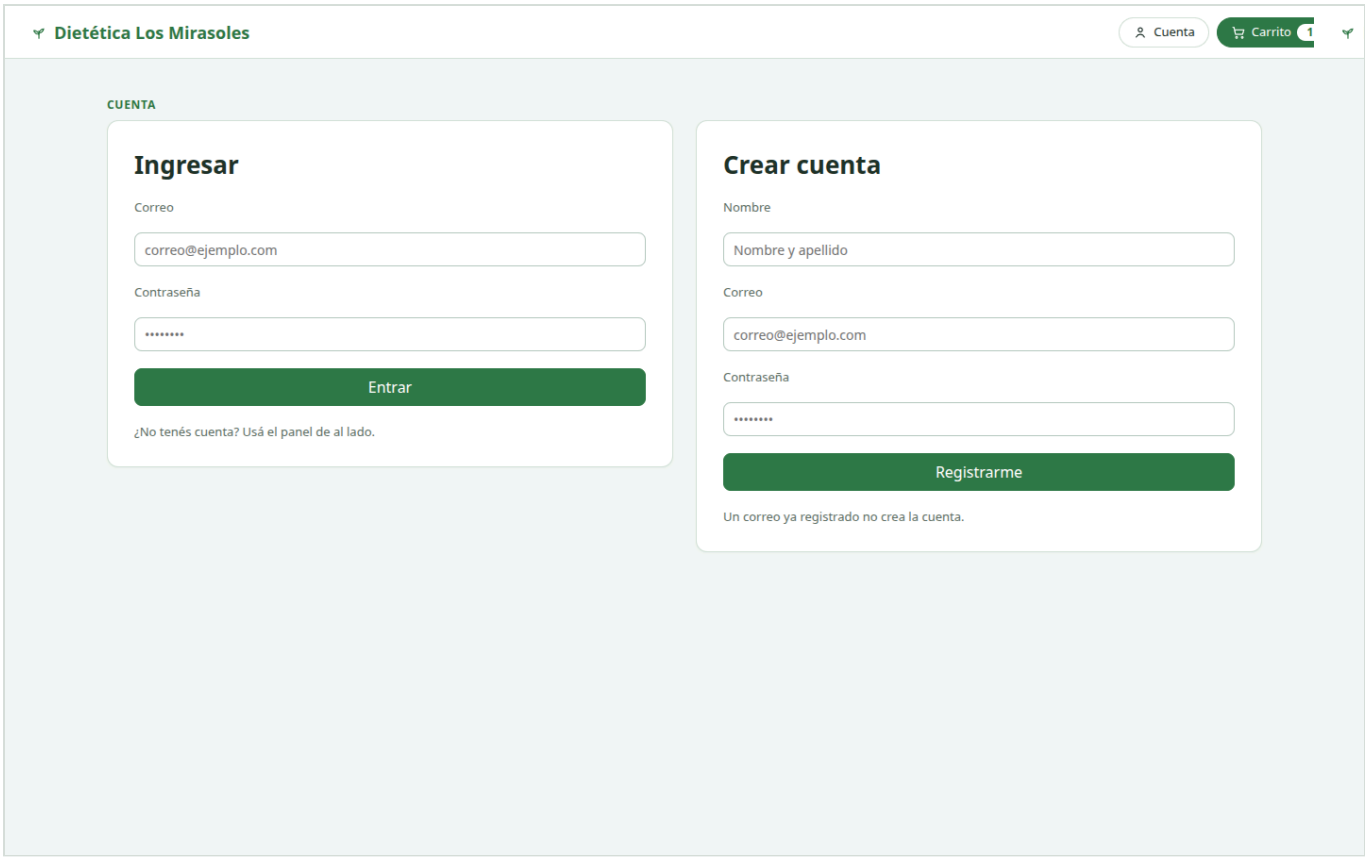


Figura 4. Pantalla de ingreso y de alta, con el carrito todavía en 1. RF-22 y RF-23.

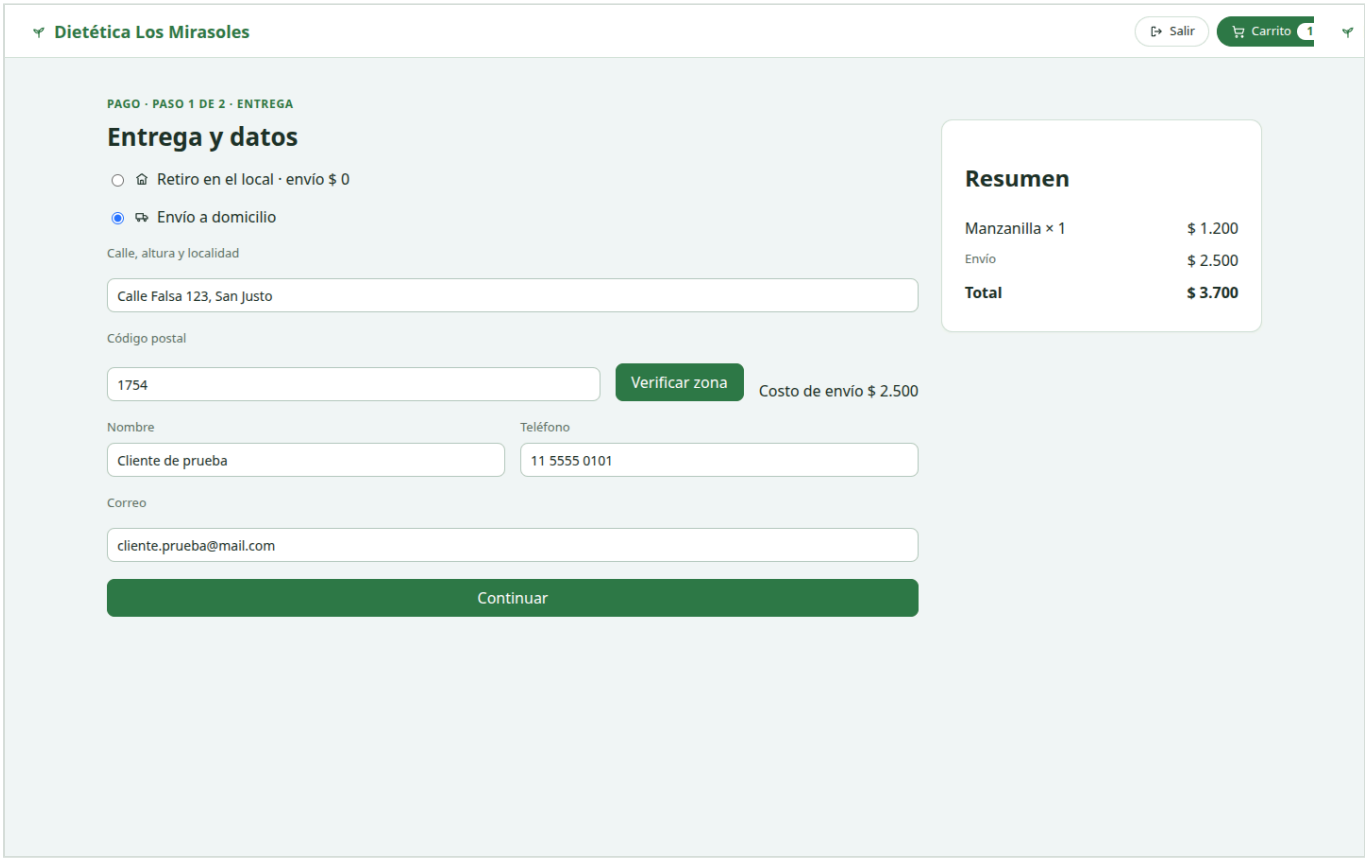


Figura 5. Con la cuenta de prueba se eligió envío a San Justo, código 1754. La zona cargada informa $ 2.500 y el total pasa a $ 3.700. RF-12, RF-13, RF-14 y RF-17.

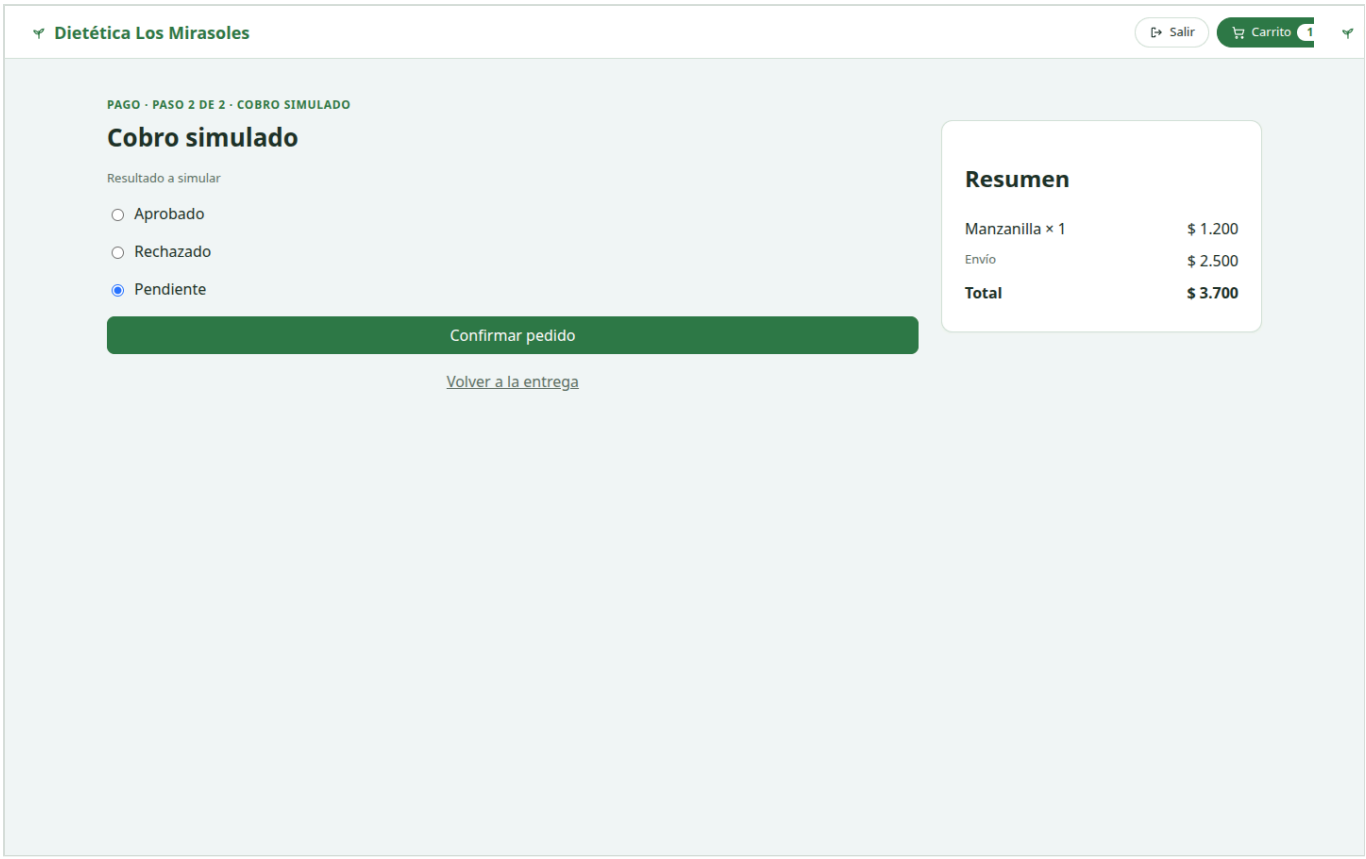


Figura 6. El segundo paso ofrece aprobado, rechazado o pendiente. No hay campo de tarjeta. Se eligió pendiente. RF-18.

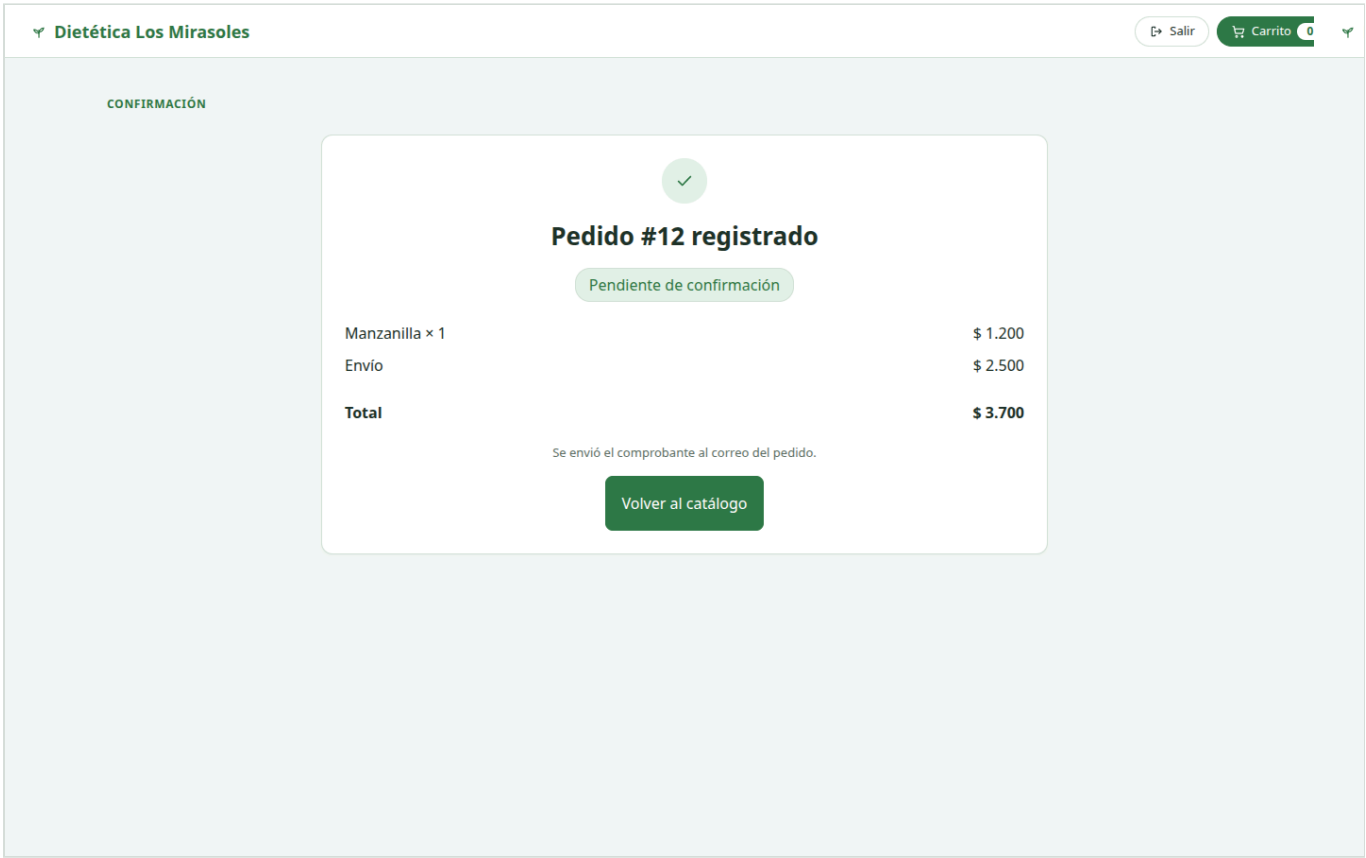


Figura 7. El pedido 12 quedó registrado, pendiente de confirmación, con el mismo total. La pantalla dice que el comprobante se envió. El carrito volvió a 0. RF-19, RF-20 y RF-21. En la base, `mail_ok` quedó en verdadero y el stock de Manzanilla siguió en 15.

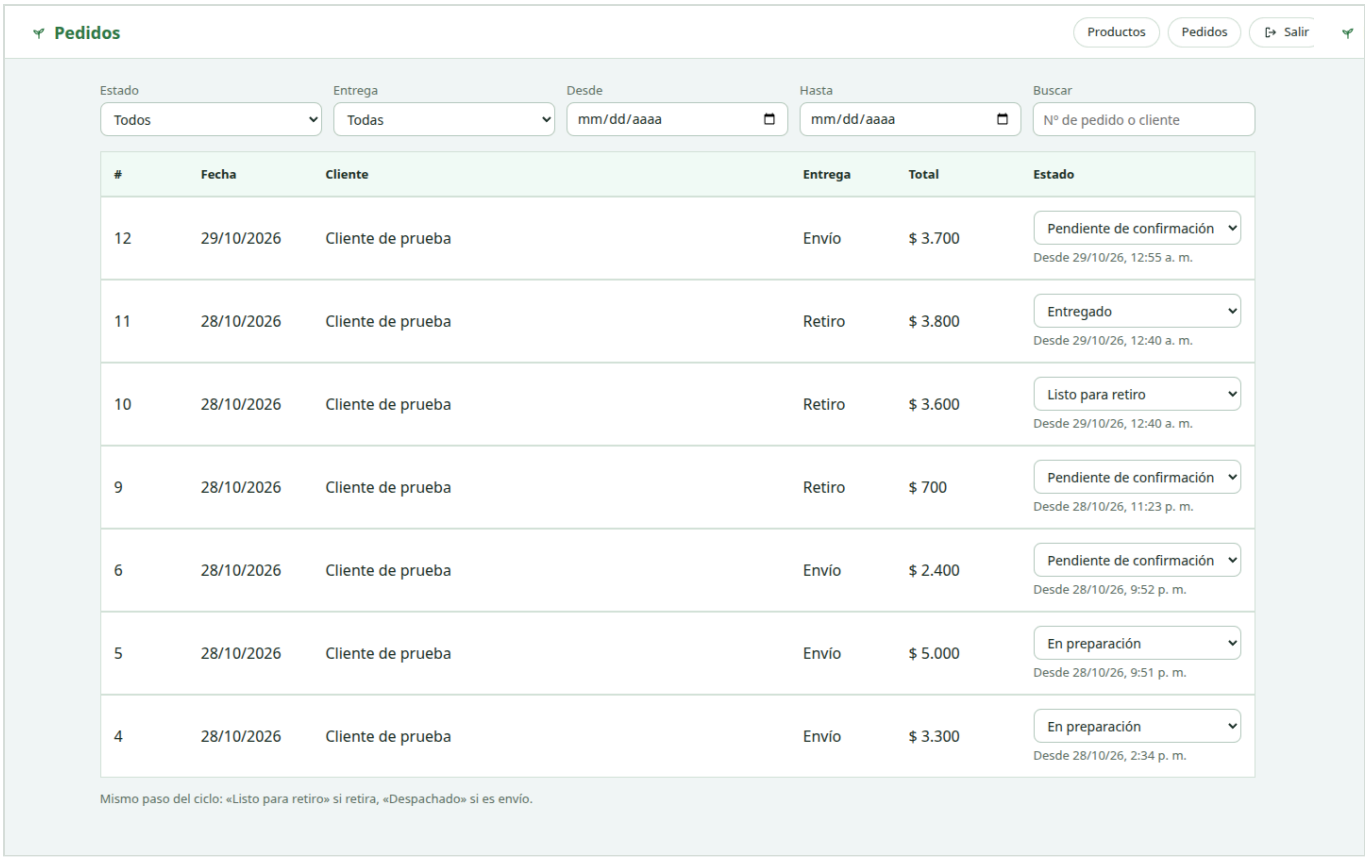


Figura 8. Con la cuenta de administrador se abrió Pedidos. El pedido 12 aparece como envío, $ 3.700, pendiente de confirmación, con filtros y búsqueda. RF-33 y RF-34. No se cambió el estado, para no mover el stock.

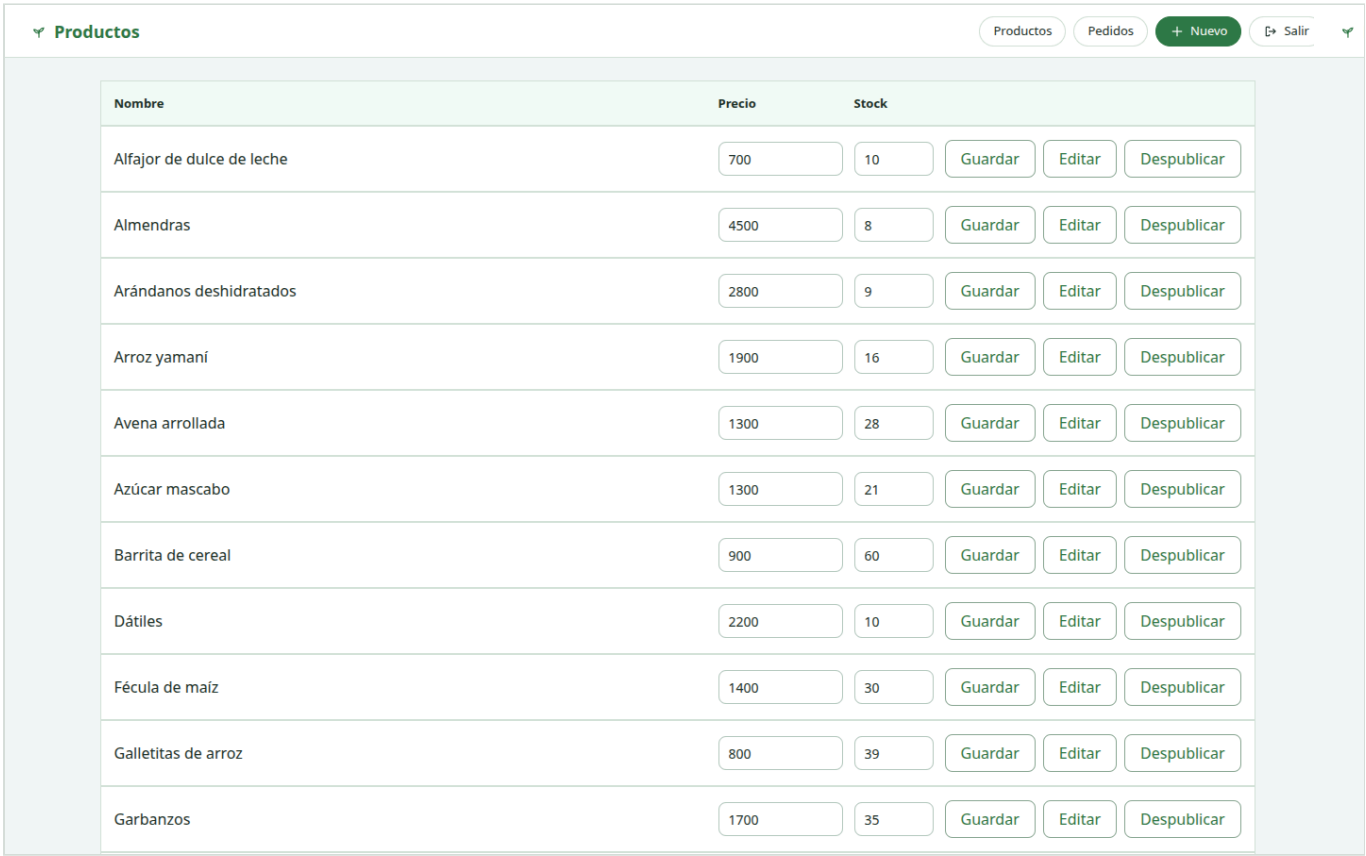


Figura 9. La sección Productos lista el catálogo con precio y stock editables en la fila, y el alta en un formulario. RF-29 y RF-31. No se guardó ni se eliminó nada en esta corrida.

Con la misma cuenta de cliente, abrir `/admin/pedidos` mostró el texto «Esta pantalla es del administrador» y el enlace «Volver al catálogo».

7. Pruebas y trazabilidad

El 29/10/2026, `npm run cy:run` ejecutó 11 pruebas y las 11 pasaron, con Node 24.13.1 y Cypress 16.2.0. Esas pruebas simulan Supabase: no crean pedidos y no prueban el panel ni el correo. La corrida manual del mismo día usó la tienda real. El pedido 12 es el resultado de esa corrida.

El detalle de pasos está en el Anexo A. La matriz usa las rutas del código entregado.

RF	Criterio	Implementación	Prueba	Estado
RF-01	Ver el catálogo publicado sin cuenta	`src/pages/Catalogo.jsx` `Catalogo`	CP-01	Parcial
RF-02	La ficha muestra los datos del producto	`src/pages/Ficha.jsx` `Ficha`	CP-02	Completo
RF-03	Las etiquetas se ven en la tarjeta y en la ficha	`src/lib/presentacion.js` `etiquetasDe`	CP-03	Completo
RF-04	Categoría y aptitud se combinan sin recargar	`src/pages/Catalogo.jsx` `productosFiltrados`	CP-04	Completo
RF-05	La búsqueda ignora mayúsculas y tildes	`src/pages/Catalogo.jsx` `normalizar`	CP-05	Parcial
RF-06	Sin stock no se puede agregar	`src/pages/Catalogo.jsx`	CP-06	Completo
RF-08	Cantidad, quita y subtotal	`src/context/CartContext.jsx` `cambiarCantidad`	CP-08	Completo
RF-11	Si no alcanza el stock, no hay pedido	`supabase/schema.sql` `confirmar_pedido`	CP-11	Completo
RF-12	Elegir retiro o envío	`src/pages/Pago.jsx` `continuar`	CP-12	Completo
RF-13	El código postal determina el costo	`supabase/schema.sql` `confirmar_pedido`	CP-13	Parcial
RF-14	El costo se ve antes de confirmar	`src/pages/Pago.jsx` `verificarZona`	CP-14	Completo
RF-17	Faltan datos: no se avanza	`src/pages/Pago.jsx` `continuar`	CP-17	Completo
RF-18	Simular el cobro sin tarjeta	`src/pages/Pago.jsx`	CP-18	Completo
RF-19	Pendiente graba y no descuenta	`supabase/schema.sql` `confirmar_pedido`	CP-19	Completo
RF-20	La confirmación muestra el número y el total	`src/pages/Confirmacion.jsx` `Confirmacion`	CP-20	Completo
RF-21	Sale el comprobante o queda el fallo	`supabase/functions/confirmar-pedido/index.ts` `sendReceipt`	CP-21	Completo
RF-22	El correo repetido no crea otra cuenta	`src/pages/Ingreso.jsx` `crearCuenta`	CP-22	Completo
RF-23	Ingresar y salir	`src/context/AuthContext.jsx` `cerrarSesion`	CP-23	Completo
RF-28	El cliente no entra al panel	`src/components/PuertaAdmin.jsx` `PuertaAdmin`	CP-28	Completo
RF-29	Alta y edición de producto	`src/pages/admin/Productos.jsx` `guardar`	CP-29	Completo
RF-31	Precio y stock en la fila	`src/pages/admin/Productos.jsx`	CP-31	Completo
RF-33	Filtrar y buscar pedidos	`src/pages/admin/Pedidos.jsx`	CP-33	Completo
RF-34	Cambiar el estado y guardar la fecha	`supabase/schema.sql` `aplicar_cambio_estado_pedido`	CP-34	Completo

8. Cierre, aportes y fuentes

Limitaciones que afectan a un requisito:

- RF-01: no hay subcategorías y el catálogo cargado tiene 32 productos.
- RF-05: la búsqueda no ordena por relevancia.
- El carrito vive en memoria. Recargar el navegador lo vacía.
- RF-13: un código postal desconocido no impide el envío.
- RF-20: la confirmación vive en la navegación. Recargar esa pantalla no muestra el pedido.
- RF-21: sin los secretos de Gmail el pedido queda y el correo no sale.
- RNF-10: el bloqueo ante dos compras simultáneas está escrito y no se ejecutó.
- RNF-13: no hay auditoría de precio y stock.
- RNF-15: el selector de fecha del panel depende del idioma del navegador.

Un cobro rechazado conserva el carrito y no crea un pedido. La pantalla no explica la causa.

Aportes, según el historial del repositorio y el trabajo de esta entrega:

- Kevin Ojea: checkout, confirmación, panel, esquema de stock, función de correo, estilos, separación del administrador y ajustes de las pruebas.
- Juan Garcia: catálogo público, filtros, búsqueda, carrito, ingreso y sesión, y las pruebas de Cypress de esas pantallas y del checkout.

Conclusión del equipo: el corte de prioridad alta se puede recorrer de punta a punta con las dos cuentas de demostración. La evaluación de la tienda depende del proyecto de Supabase ya cargado.

Las fotos del catálogo fueron generadas para este trabajo y están en el depósito de Supabase; no se copian en el archivo de código. No se reutilizó un repositorio de terceros como base de la tienda. Supabase, React, Vite y Cypress son las dependencias declaradas en `package.json`.

Se usó Claude y Cursor.

Anexo A. Casos de prueba

CP-01 · RF-01

Precondición: sin sesión. Pasos: abrir `/`. Esperado: se ve el catálogo publicado. Obtenido: se vio el catálogo y las categorías, figura 1. No hay subcategorías. Aprobada en lo que la entrega incluye. Evidencia: figura 1.

CP-02 · RF-02

Pasos: abrir Manzanilla. Esperado: ficha con los datos del requisito. Obtenido: imagen, etiquetas, precio, unidad «paquete 40 g», descripción y nutrición, figura 2. Aprobada. Evidencia: figura 2.

CP-03 · RF-03 y CP-04 · RF-04

Precondición: Cypress con catálogo simulado. Pasos: los de `cypress/e2e/catalogo.cy.js`, filtros y aptitud. Esperado: los filtros se combinan sin otro pedido de productos, y las etiquetas se distinguen. Obtenido el 29/10/2026: las dos pruebas del archivo pasaron. Aprobadas.

CP-05 · RF-05

Esperado: «organica» encuentra «Orgánica» y el orden es por relevancia. Obtenido: no se tipeó la búsqueda en esta corrida manual. El código de `normalizar` saca mayúsculas y tildes, y el filtro no reordena. Parcial. La parte de relevancia no está implementada.

CP-06 · RF-06

Esperado: un producto sin stock sigue visible y no tiene «Agregar». Obtenido: la prueba de `catalogo.cy.js` pasó el 29/10/2026. En el catálogo real, Mix de semillas y Manzana deshidratada se ven como «Sin stock». Aprobada.

CP-08 · RF-08

Pasos: agregar Manzanilla. Esperado: cantidad y subtotal. Obtenido: una unidad y $ 1.200, figura 3. Las pruebas de `carrito.cy.js` (tope, resta, quita y vaciado) pasaron el mismo día. Aprobada.

CP-11 · RF-11

Esperado: si el stock no alcanza, no hay pedido y el carrito sigue. Obtenido: `checkout.cy.js` simuló el 409 y pasó. No se agotó un producto real durante la corrida manual. Aprobada la prueba simulada. La simultaneidad de RNF-10 no se ejecutó.

CP-12 · RF-12, CP-13 · RF-13 y CP-14 · RF-14

Datos: Calle Falsa 123, San Justo, código 1754, cuenta `cliente.prueba@mail.com`. Esperado: elegir envío, ver el costo de la zona y sumarlo al total. Obtenido: envío $ 2.500 y total $ 3.700, figura 5. El caso negativo de un código inexistente no se repitió en esta corrida; el código lo cobra a 0 y sigue. CP-12 y CP-14 aprobadas. CP-13 parcial.

CP-17 · RF-17

Esperado: sin un dato obligatorio no se pasa al cobro. Obtenido: el formulario exige los campos y, con los datos completos, se pasó al paso 2. El caso negativo de Cypress, que deja un campo vacío, pasó el 29/10/2026. Aprobada.

CP-18 · RF-18

Esperado: tres resultados y ningún dato de tarjeta. Obtenido: figura 6. Aprobada.

CP-19 · RF-19 y CP-20 · RF-20

Pasos: confirmar con resultado pendiente. Esperado: hay pedido, el estado espera confirmación, el carrito se vacía y el stock no cambia. Obtenido: pedido 12, figura 7. En la base, estado `pendiente_confirmacion`, total 3700, stock de Manzanilla 15 antes y después. Aprobadas. Evidencia: figura 7 y la fila del pedido 12.

CP-21 · RF-21

Esperado: el comprobante sale o el fallo queda registrado. Obtenido: la confirmación dijo que el correo se envió y `pedidos.mail_ok` del pedido 12 quedó en verdadero. Aprobada. No se adjunta el mensaje porque contiene el correo de la cuenta de prueba y no hace falta para ver el flag.

CP-22 · RF-22

Esperado: un correo ya registrado no crea otra cuenta y lo dice. Obtenido: `ingreso.cy.js` pasó el 29/10/2026. Aprobada. El alta exitosa del mismo archivo también pasó, con datos inventados por la prueba.

CP-23 · RF-23

Pasos: entrar con la cuenta de prueba y después salir. Esperado: la sesión abre la tienda y al salir vuelve «Cuenta». Obtenido: después de entrar se vio «Salir» y el checkout; al salir, el encabezado volvió a «Cuenta».

Aprobada.

CP-28 · RF-28

Pasos: con la cuenta de cliente, abrir `/admin/pedidos`. Esperado: aviso de acceso y ningún listado. Obtenido: «Esta pantalla es del administrador» y «Volver al catálogo». Aprobada. Con la cuenta de administrador, la misma ruta mostró el listado, figura 8.

CP-29 · RF-29 y CP-31 · RF-31

Esperado: un formulario de alta y precio y stock en la fila. Obtenido: figura 9. No se guardó un cambio ni se eliminó un producto, para no alterar el catálogo de la demostración. La presencia de las acciones está verificada. La escritura en la base no se ejecutó en esta corrida.

CP-33 · RF-33 y CP-34 · RF-34

Esperado: el pedido nuevo se ve con su estado y su fecha. Obtenido: el pedido 12, pendiente, «Desde 29/10/26, 12:55 a. m.», figura 8. No se tipeó una búsqueda ni se cambió el estado en esta corrida. El cambio de estado y la devolución de stock están en el trigger y se usaron en pedidos anteriores del mismo proyecto; no se repitieron acá para no modificar el pedido de la evidencia.